Date	Version	Description
20/06/2026	1.0	• Création de l'énoncé

I. Pré-requis

- Visual Studio 2022 ou 2026 avec la charge de travail Développement web et ASP.NET.
- Le SDK .NET 9 ou 10 (installé automatiquement avec VS)
- Docker Desktop (installation)
- DBeaver - facultatif (téléchargement)

II. Préambule

Le but de ce TP va être de mettre en place le squelette d'une API C# à partir du modèle Microsoft. Puis d'implémenter les premières routes afin d'effectuer un CRUD.

III. Créer la solution et le projet

Appuyez vous sur la **"Notice d'installation globale"** afin de créer le projet

❓ Une fois dans le projet essayer de retrouver le rôle des fichiers suivant :

- Program.cs
- appsettings.json
- launchSettings.json

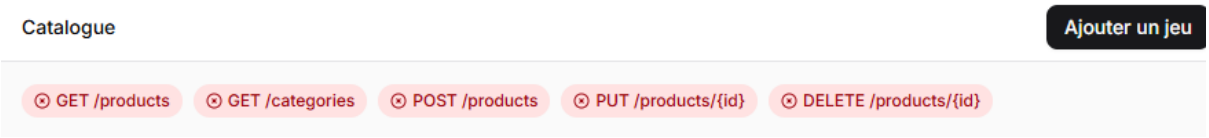
Lancer l'application générée et essayer de joindre la route donnant les prévisions météo.

*Rq : Une solution (fichier *.sln) représente un regroupement de projets (fichiers *.csproj) cela permet de découper une application en plusieurs briques (dépendantes ou non).*

IV. Créer une première route

Inspirez vous de WeatherForecastController.cs pour créer votre propre controller qui permette de répondre aux routes du catalogue du projet front.

Les routes appelées par le front sont présentées sous forme de badge dans la page catalogue.



Au clic sur chaque badge vous aurez à disposition les informations attendues par le front.

Il vous faudra créer :

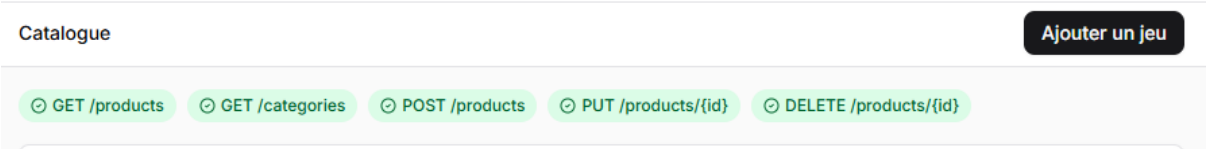
- Le controller
- Les modèles de réponse

Les modèles de réponse seront créés directement dans le controller pour le moment avec des données statiques.

💡 Tips :

- Le controller doit hériter du type ControllerBase
- Pour les modèles de réponse, s'appuyer sur les classes attendues par le front

Vous devez être capable via le front de consulter le catalogue, créer, modifier et supprimer un article. Et si tout se passe bien :



V. Connection à la base de données

Maintenant il va falloir connecter une base de données pour conserver les données malgré un redémarrage de l'application.

La base de données sera gérée en "Code First" via la librairie Entity Framework.

💡 Appuyez vous sur la notice **"Notice - Entity Framework"** pour l'installation et la configuration de la librairie.

Ensuite pour créer les entités qui donneront les tables et leur relation, voici ce que le métier vous dit, à vous d'en faire des classes :

"FTELMarket est une place de marché (type Leboncoin ou Vinted). Et chaque utilisateur possède un catalogue de produits. Un produit porte un nom, une description facultative, un prix, une quantité en stock, et on garde la date à laquelle il a été créé dans le catalogue.

Chaque produit est rangé dans une catégorie, et une seule. Une catégorie a un nom et regroupe plusieurs produits."

Rien d'autre n'est imposé. Les noms, les types, l'emplacement des fichiers : ce sont vos décisions, et vous devrez pouvoir les justifier.
Les entités en base de données ne doivent pas forcément reprendre le même modèle que les classes créées dans le controller.

❓ Questions à se poser :

- Le prix : decimal, double ou float ? La réponse n'est pas indifférente quand il s'agit d'argent — cherchez pourquoi.
- Les dates : Comment faire si votre application est utilisée sur plusieurs fuseaux horaires ?

VI. Appliquer le modèle de données sur la BDD

S'appuyer sur la notice **"Notice - Commandes Entity Framework"** pour créer et appliquer les migrations sur la base de données.
Vérifier directement en base de données la bonne création des tables en base de données.

VII. Flow complet : Du navigateur à la DB

Assurer la connexion entre les méthodes du controller et la base de données.
On doit pouvoir lire, ajouter, modifier, supprimer, ajouter une entrée du catalogue et lister toutes les entrées du catalogue.